Banking API Assessment — Complete Code Changes & Architecture Guide

Technical Audit & Interview Presentation Document

37 / 37 TESTS PASSING COMPLETE CORE + STRETCH

1. Master File Change Matrix

File Path	Status	Layer	Work Performed & Technical Purpose
`src/handlers/customers.js`	MODIFIED	Business Logic	Replaced 501 stubs. Implemented `POST /customers` (validation, UUID, status ACTIVE) and `GET /customers/{id}` (O(1) point read).
`src/handlers/accounts.js`	MODIFIED	Business Logic	Replaced 501 stubs. Implemented `POST /accounts` (parent customer check, currency/type validation), `GET /accounts` via **GSI CustomerIdIndex Query**, and `GET /accounts/{id}`.
`src/handlers/transactions.js`	MODIFIED	Business Logic (Stretch)	Replaced 501 stubs. Implemented atomic deposits, overdraft-protected withdrawals, and atomic cross-account transfers using **DynamoDB ACID TransactWriteItems**.
`src/lib/response.js`	MODIFIED	Core Utility	Hardened `parseJsonBody` to support strings, base64, and pre-parsed objects. Enforced `{ error, message }` OpenAPI error schemas.
`serverless.yml`	MODIFIED	Configuration	Added root `/` route mapping to `docsUi` so opening `http://localhost:3000` loads Swagger UI directly.
`package.json`	MODIFIED	Configuration	Added `dev`, `test`, `db:local`, and `screenshots` scripts. Added `aws-sdk-client-mock` and `dynalite` devDependencies.
`README.md`	MODIFIED	Documentation	Added command quick-reference table, architecture diagrams, and documentation links.
`.env`	CREATED	Configuration	Configured environment variables for local DynamoDB connection on `http://localhost:8000`.
`test/customers.test.js`	CREATED	Automated Test	9 tests covering customer creation, missing fields, email regex validation, and 404 not found handling.
`test/accounts.test.js`	CREATED	Automated Test	13 tests covering account creation, initial balance precision, currency validation, 404s, and GSI queries.
`test/transactions.test.js`	CREATED	Automated Test	15 tests covering deposits, withdrawals, overdraft protection (409), atomic transfers, and concurrency collisions.
`scripts/dev.js`	CREATED	Dev Experience	1-command startup: auto-cleans stale ports (8000/3000/3002), auto-copies `.env`, starts local DB, seeds data, and starts API.
`scripts/start-local-db.js`	CREATED	Infrastructure	Pure Node.js in-memory DynamoDB Local server using `dynalite` (eliminates Docker Desktop requirement).

File Path	Status	Layer	Work Performed & Technical Purpose
`scripts/create-tables.js`	**MODIFIED**	Migration	Hardened table recreation logic with backoff delays and `ResourceInUseException` resilience.
`scripts/capture-all-screenshots.js`	**CREATED**	QA / Verification	Runs end-to-end tests for all 9 API operations against localhost:3000 and captures high-res PNG screenshots.
`NOTES.md`	**CREATED**	Documentation	Complete technical documentation on DynamoDB access patterns, ACID transactions, and curl demo commands.
`Banking_API_Interview_Guide.pdf`	**CREATED**	Interview Prep	Printable PDF guide with elevator pitches, model answers to interviewer questions, and cheat sheets.

2. Detailed Architecture & Code Implementation

1. Customers Handlers (`src/handlers/customers.js`)

Replaced initial 501 stubs with production handlers:

- `create(event)` :
  - Parses request body via `parseJsonBody` .
  - Validates `fullName` (non-empty string) and `email` (RFC 5322 regex pattern).
  - Generates unique ID with `uuidv4()` prefixed as `cust_<uuid>` .
  - Sets status `ACTIVE` and ISO timestamp `createdAt` .
  - Persists to `Customers` table with `PutCommand` and returns `201 Created` .
- `get(event)` :
  - Performs direct $O(1)$ point lookup using `GetCommand` by partition key `customerId` .
  - Returns `200 OK` with customer item or `404 Not Found` with `{ error: "NOT_FOUND", message: "..." }` .

2. Accounts Handlers (`src/handlers/accounts.js`)

Implements customer account lifecycle and optimized query patterns:

- `create(event)` :
  - Validates parent customer exists via `GetCommand` (returns `404` if not found).
  - Validates `type` (`CHECKING` or `SAVINGS`) and `currency` (`CAD` or `USD`).
  - Validates `initialBalance` : must be a non-negative finite number with at most 2 decimal places.
  - Persists to `Accounts` table with partition key `accountId` and indexed attribute `customerId` . Returns `201 Created` .
- `list(event)` — *Crucial Optimization*:
  - Verifies customer exists first (returns `404` if not).
  - **Query vs Scan:** Instead of executing a table-wide Scan with a filter, uses `QueryCommand` on the `CustomerIdIndex` **GSI**.
  - Targets only that customer's partition in $O(1)$ time, saving read capacity units and scaling sub-linearly. Returns `200 OK` with `{ items: Account[] }` .
- `get(event)` : Point read by `accountId` returning current account data including balance (200 / 404).

3. Transactions Handlers (`src/handlers/transactions.js`) — Stretch Goals

Provides atomic balance operations and ACID fund transfers:

- `deposit(event)` :
  - Validates amount (> 0, max 2 decimals) and matching currency. Verifies account is `OPEN` .
  - Atomic balance increment via `UpdateCommand` with `SET balance = balance + :amount` .
  - Condition guard: `attribute_exists(accountId) AND #status = :openStatus AND currency = :currency` . Returns `200 OK` .
- `withdraw(event)` :
  - Validates amount, currency match, and `OPEN` status. Rejects if balance < amount with `409 Conflict` .
  - Atomic debit via `UpdateCommand` : `SET balance = balance - :amount` .
  - Concurrency guard: `ConditionExpression: "balance >= :amount AND #status = :openStatus AND currency = :currency"` .
  - Catches `ConditionalCheckFailedException` to safely reject concurrent race collisions with `409 Conflict` .
- `transfer(event)` :
  - Validates both accounts exist, are `OPEN` , share currency, and `fromAccountId !== toAccountId` .
  - **ACID Atomicity:** Executes debit and credit together in a single `TransactWriteCommand` (`TransactWriteItems`). Either both operations commit or both roll back.
  - Debit condition: `balance >= :amount` . If concurrent withdrawals deplete the balance, DynamoDB aborts the transaction with `TransactionCanceledException` , caught and returned as `409 Conflict` .

3. Automated Test Suite (37 Tests Across 11 Suites)

Test Suite File	Tests	What is Tested
`test/customers.test.js`	**9 Tests**	Happy path customer creation, UUID generation, status ACTIVE, missing fullName (400), whitespace fullName (400), invalid email regex (400), missing body (400), malformed JSON (400), point retrieval (200), missing customer (404), missing path param (400).
`test/accounts.test.js`	**13 Tests**	Creation with default initialBalance=0 (201), custom initialBalance (201), non-existent customer rejection (404), invalid type (400), invalid currency (400), negative balance (400), fractional cents >2 decimals (400), GSI query execution (200), empty items array (200), listing non-existent customer (404), account retrieval by ID (200), missing account (404), missing path param (400).
`test/transactions.test.js`	**15 Tests**	Deposit happy path (200), deposit non-existent account (404), non-positive amount (400), currency mismatch (400), closed account (400), withdrawal happy path (200), insufficient funds (409), race condition simulation (409), missing account (404), atomic transfer happy path (200), self-transfer rejection (400), missing source/dest (404), insufficient funds transfer (409), currency mismatch (400), transaction collision rollback (409).

4. Developer Experience & Local Infrastructure Scripts

- `scripts/dev.js` **(Unified Dev Startup):**
  - Allows the entire environment to run via `npm run dev` in a single command.
  - Auto-frees ports 8000, 3000, and 3002 before launch, completely eliminating `EADDRINUSE` errors.
  - Auto-creates `.env` from `.env.example` if missing.
  - Spawns local DynamoDB, seeds tables, and starts serverless-offline.
- `scripts/start-local-db.js` **(In-Memory DynamoDB):**
  - Runs an in-memory DynamoDB server using `dynalite`, meaning reviewers don't even need Docker Desktop installed.
- `scripts/capture-all-screenshots.js` **(Automated Visual QA):**
  - Runs all 9 endpoints against localhost:3000, generates visual HTML reports, and captures PNG screenshots via headless Microsoft Edge.

5. How to Explain Your Changes in an Interview

> ***Elevator Summary to Give the Interviewer:***
> *"I replaced the initial 501 stubs across all three handlers—customers, accounts, and transactions—with production-ready logic. I ensured strict compliance with the OpenAPI 3.0 specification for all status codes and error shapes.*
>
> *For data access, I optimized customer account queries by using a Global Secondary Index (GSI) rather than table Scans. For financial transfers, I used DynamoDB TransactWriteItems to guarantee ACID atomicity across accounts, combined with optimistic concurrency condition expressions to prevent overdrafts and race conditions.*
>
> *Finally, I verified the entire implementation with a 37-test automated test suite using Node's native test runner and aws-sdk-client-mock, and built a unified 1-command startup script for seamless local execution."*